

Author: Ilaria Teberneac

1) The Fibonacci Problem

The naive Fibonacci belongs to the exponential complexity class $O(2^n)$. Its code is shorter than the memoized version, and I felt it was easier to code. Each call makes two more calls, so the call tree doubles at almost every level. The same values are computed again and again, which makes the whole process more complex and thus inefficient. The time went up about 1.5 times every time n went up by 1, so every 10 steps of n made the naive Fibonacci more than 100 times slower. At $n = 10$ it made 109 calls and took 0.000018 seconds, at $n = 20$ it took 0.001693 seconds, and at $n = 30$ it made 1,664,079 calls and took 0.191848 seconds. At $n = 40$ it made 204,668,309 calls and took 22.65 seconds. Because of this fast growth, going much past $n = 40$ would take minutes or even hours to run on my computer.

The quick Fibonacci belongs to the linear complexity class $O(n)$. Its values are memoized, so they don't have to be recomputed. The first time it computes a value, it saves it in the cache. The next time it needs that value, it just looks it up instead of making more calls, which makes it very efficient. The first thing I noticed is that, compared to the previous simulation, there were far fewer calls. At $n = 20$ it made 37 calls, at $n = 30$ it made 57 calls, and at $n = 40$ it made 77 calls, so +20 calls per each 10 steps. Also to mention that it took only 0.000016 seconds in the last step (at 40). This is significantly less than the naive Fibonacci, which made hundreds of millions of calls at $n = 40$. This shows that saving values in a cache turns an exponential class into a linear one.

2) Sorting

Selection sort belongs to the quadratic complexity class $O(n^2)$. Its code is short and simple, because it searches for the smallest value and moves it to the front, one position at a time. However, for every position in the list, it searches through the rest of the list to find the smallest value, which makes it more complex, and it requires energy and memory even for work it has already done. For example, on the sorted list of 256 numbers it still made 32,896 comparisons and only 252 swaps, which is almost equal to the number of values in the list. Every time the list size doubled, the time went up around 4 times, so for 32 numbers it needed 0.000070 seconds, while for 16,384 numbers it took 13.61 seconds, which is not time-efficient.

Quick sort belongs to the $O(n \log n)$ linearithmic complexity class. Each step selects a pivot and shifts the smaller values to its left and the larger values to its right. Compared to selection sort, it was much more time-efficient. Every time the list size doubled, the time only went up with around 2 times. At 32 numbers it took 0.000033 seconds, while at 16,384 numbers it took 0.047972 seconds, which is significantly less time per number than selection sort. Also, compared to selection sort, there are many more swaps here: for the same list size of 252, I got 1,406 swaps, but it is still more productive because it requires fewer comparisons.